

PROCEDURES STOCKEES EN T-SQL

Principes

Une procédure stockée est une suite d'instructions qui vont avoir des effets sur la base de données ou qui renvoient des valeurs.

Les procédures stockées sont rattachées aux bases de données SQL Server et deviennent des objets de la base de données, tout comme les tables et les vues.

L'intérêt principal est de concentrer les traitements sur les données au niveau du SGBD. Ainsi, lorsque des mises à jour importantes doivent être effectuées sur les données, il est souvent plus aisé de réaliser les modifications par procédure stockée que via une programmation extérieure dans un langage de programmation classique.

Si vous effectuez des calculs qui impliquent un grand nombre de lignes, vous pouvez éviter de télécharger trop d'informations vers le client en définissant une procédure stockée pour effectuer le travail sur le serveur. Les Procédures Stockées sont exécutées plus rapidement car elles sont compilées et elles n'ont pas besoin de déplacer les données du serveur vers le client.

Création d'une procédure stockée

Dans SQL, pour créer une procédure, vous démarrez avec l'expression **CREATE PROCEDURE**. Vous pouvez également utiliser **CREATE PROC**. Les deux expressions produisent le même résultat.

Après le nom de la procédure, tapez le mot-clé **AS**. La section, le groupe de mots, ou un groupe de lignes après le mot-clé **AS** est appelé le corps de la procédure. Il énonce ce que vous voulez que la procédure fasse ou ce que vous souhaitez produire.

Sur cette base, la syntaxe générale de l'instruction est la suivante :

```
CREATE PROCEDURE procedure_name  
AS  
Body of the procedure
```

Exécution d'une procédure stockée

Les procédures stockées étant destinées à être utilisées par des applications clientes, il n'existe pas de moyen pour les exécuter et voir le résultat retourné sans passer par une application Client. Dans le cadre de ce cours, on utilisera SQL Server Management Studio pour tester les procédures stockées créées.

Pour exécuter une procédure stockée les instructions **execute** et **exec** (format réduit de execute) peuvent être utilisées.

Remarques

- Le corps de la procédure créée peut inclure n'importe quelle instruction excepté Create Procedure, Create trigger, Create View, Create Rule et Create Default ;
- Une procédure stockée peut utiliser des **tables temporaires locales** (tables dont le nom commence par # qui sont créées dans la procédure et ne sont reconnues que dans cette procédure et dans les procédures appelant cette procédure) et des **tables temporaires globales** (dont le nom commence par ##).

Procédure sans paramètre

La procédure stockée exécute un traitement donné mais ce traitement ne dépend d'aucune valeur provenant de l'application appelante.

Syntaxe :

```
CREATE PROCEDURE Nom_Procédure AS  
Instructions
```

Exécution :

```
EXEC Nom_Procedure
```

Exemple :

- Créer une procédure stockée qui calcule le nombre de commandes par client :

USE Northwind

```
IF EXISTS(SELECT name FROM sysobjects WHERE name =  
'Nombre_de_commandes_par_client')
```

```
DROP PROCEDURE Nombre_de_commandes_par_client
```

GO

```
CREATE PROCEDURE Nombre_de_commandes_par_client  
AS
```

```
SELECT CustomerID, COUNT(*) AS Nbre  
FROM Orders  
GROUP BY CustomerID
```

GO

EXEC Nombre_de_commandes_par_client

GO

Procédure avec des paramètres en entrée

La procédure stockée en fonction de valeurs provenant de l'extérieur va effectuer certains traitements. Il n'est pas donc cohérent qu'une procédure stockée reçoive des paramètres en entrée dont les valeurs ne soient pas exploitées dans les instructions des procédures (dans des tests, dans des conditions...).

Syntaxe :

```
CREATE PROCEDURE Nom_Procedure  
    Nom_Param1_Entrée Type_Donnée = Valeur_Par_Default,  
    Nom_Param2_Entrée Type_Donnée = Valeur_Par_Default...  
AS  
    Instructions
```

Exécution :

```
Exec Nom_Procedure Valeur_Param1, Valeur_Param2...
```

Ou

```
Exec Nom_Procedure Nom_Param1 = Valeur_Param1,  
    Nom_Param2 = Valeur_Param2...
```

Remarque :

Avec la deuxième syntaxe, l'utilisateur n'est pas obligé de passer les paramètres dans l'ordre et en plus si des paramètres ont des valeurs par défaut, il n'est pas obligé de les passer.

Exemple :

Créer une procédure stockée qui affiche la liste des commandes d'un client dont le numéro est passé en paramètre :

USE Northwind

IF EXISTS(SELECT name FROM sysobjects WHERE name =
'Commandes_client')

DROP PROCEDURE Commandes_client

GO

CREATE PROCEDURE Commandes_client @CustomerID NCHAR(5)

AS

```
SELECT CustomerID, OrderID, OrderDate  
FROM Orders  
WHERE CustomerID=@CustomerID
```

GO

On peut exécuter cette procédure pour afficher la liste des commandes du client dont le customerID est 'ALFKI' :

```
EXEC Commandes_client 'ALFKI'  
GO
```

Ou

```
DECLARE @CustomerID NCHAR(5)  
SET @customerID='ALFKI'  
EXEC Commandes_client @customerID
```

Procédure avec des paramètres en sortie

La procédure stockée, suite à un traitement réalisé, va attribuer des valeurs à des paramètres en sortie. Les valeurs de ces paramètres peuvent être récupérées par des applications clientes. Il n'est pas donc cohérent qu'une procédure stockée contenant des paramètres de sortie n'affecte pas de valeurs à ces paramètres avant la fin du traitement.

Syntaxe :

```
CREATE PROCEDURE Nom_Procedure  
    Nom_Param1_Entrée Type_Donnée = ValeurParDefaut,...  
    Nom_Param1_Sortie Type_Donnée Output,  
    Nom_Param2_Sortie Type_Donnée Output...  
AS  
    Instructions
```

Exécution :

```
Declare Var_Param1_Sortie Type_Param1_Sortie  
Declare Var_Param2_Sortie Type_Param2_Sortie  
...  
Exec Nom_Procedure Val_Param1_Entrée, Val_Param2_Entrée...,  
    Var_Param1_Sortie Output, Var_Param2_Sortie Output...
```

Exemple :

Créer une procédure stockée qui retourne le nombre total de commandes :

```
USE Northwind  
IF EXISTS(SELECT name FROM sysobjects WHERE name =  
'Nombre_total_de_Commandes')  
    DROP PROCEDURE Nombre_total_de_Commandes  
GO  
  
CREATE PROCEDURE Nombre_total_de_Commandes @Nbre INT OUTPUT  
AS  
    SET @Nbre = (SELECT COUNT(*) FROM Orders)  
GO
```

Exécution de la procédure pour afficher le nombre de commandes :

```
Declare @Nbre INT
```

```
EXEC Nombre_total_de_Commandes @Nbre OUTPUT  
PRINT 'Le nombre de commandes est de ' + CONVERT(VARCHAR,@Nbre)
```

Procédure avec valeur de retour

L'instruction return arrête l'exécution d'une procédure stockée. Une valeur entière représentant, en général, l'état d'exécution d'une procédure peut être associée à l'instruction return. Cette valeur peut être récupérée par le programme appelant de la procédure stockée.

Syntaxe :

```
CREATE PROCEDURE Nom_Procedure  
    ...  
AS  
    Instructions  
RETURN Valeur_Sortie
```

Exécution :

```
Declare Var_Retour Type_Var_Retour  
...  
Exec Var_Retour=Nom_Procedure ...
```

Exemple :

Créer une procédure stockée qui renvoie un code de retour. Si le nombre de commandes est supérieur à 500, la procédure renvoie 1. Si le nombre de commandes est inférieur à 500, la procédure renvoie 2. Si une erreur système a lieu, la procédure renvoie 3 :

```
USE Northwind  
IF EXISTS(SELECT name FROM sysobjects WHERE name =  
'Nombre_Commandes')  
    DROP PROCEDURE Nombre_Commandes  
GO  
CREATE PROCEDURE Nombre_Commandes AS  
    Declare @NbrCom int  
    Set @NbrCom = (Select count(*) from Orders)  
    If @NbrCom >=500  
        Return 1  
    If @NbrCom <500  
        Return 2  
    If @@ERROR <>0  
        Return 3  
GO
```